

黄佳 - ClaudeCode实践 - 第1章 登高望远：Claude Code技术架构全景

高屋建瓴，方知全局之妙。

小冰最近主导了一个AI Agent项目，并选用Claude Code作为核心开发工具。起初，其表现令人惊艳：只需要清晰阐述需求，代码便能迅速生成，效率远超人工手写。然而两周后，一系列问题集中爆发：每次开启新会话，Claude似乎无法记忆项目技术栈（如FastAPI+SQLAlchemy），导致开发者不得不反复重申背景信息；生成的代码风格缺乏一致性，Black格式化标准时有时无，致使PR（Pull Request，拉取请求）审查3次被回退。更为严峻的是，在执行一个涉及6个文件的重构任务时，受限于上下文窗口容量，Claude在处理至第四个文件时“遗忘”了前序逻辑，导致最终产出的代码前后矛盾，难以自治。

“感觉就像在带一个每天都会失忆的实习生，”小冰向小雪和咖哥抱怨，“能力确实不错，但什么背景信息都记不住。”

咖哥并未急于安慰，而是连发3问：

“第一，你的项目根目录配置CLAUDE.md了吗？

第二，那涉及6个文件的重构任务，为什么不拆解为子智能体协同处理？

第三，既然代码规范如此重要，为什么仅靠口头告知，而不是将其封装成Skill（技能），让模型自动加载？”

小冰哑口无言，一个问题也答不上来。

“问题根源并非Claude模型不够聪明，”咖哥指出，“而是你仅挖掘了它10%的能力——基础的对话能力。事实上，Claude Code本身就是一个完备的Agent框架，内置了记忆系统、Skills、子智能体、Hooks及MCP等。你所遭遇的每一个痛点，在框架层面均有对应的解决方案，只是你尚未知晓。”

.....

这段对话持续了整整两个小时。临近结束，小冰感慨道：“原以为自己在驾驭一个AI助手，没想到仅仅是在使用一个高级聊天窗口。”

1.1 从命令行助手到Agent框架

小冰所遭遇的Claude Code的记忆丢失、风格飘忽以及长任务上下文溢出等问题，几乎是所有将之应用于实际工程的开发者都会碰到的瓶颈。这些现象看似是3个独立的bug（缺陷），实则指向同一个根本症结：误将一个Agent框架当作简单的聊天工具来使用。

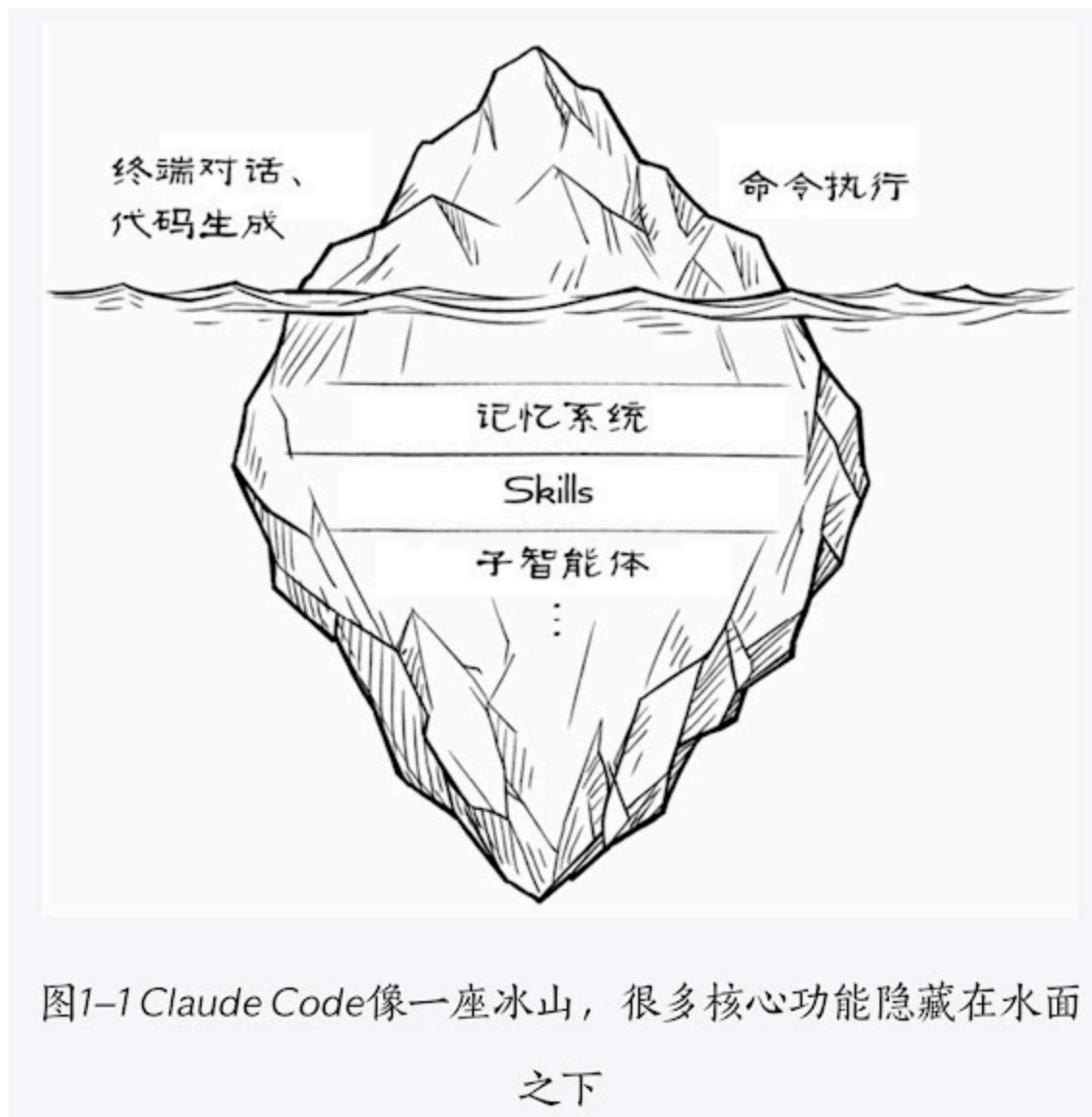
Claude Code的本质，是一个可编程、可扩展、可组合的Agent框架。

这绝非空洞的营销用语，而是其对应Claude Code核心的工程能力。

- **可编程**：意味着你可以使用Python或TypeScript代码直接驱动Claude Code，将其无缝嵌入CI/CD流水线或自动化脚本中，彻底解放人力，不需要人工值守终端。
- **可扩展**：意味着你能够通过配置文件为其注入记忆系统、Skills、Hooks等能力，而不需要触碰或修改其核心代码。
- **可组合**：意味着这些扩展模块之间可以灵活编排，如同乐高积木般自由拼装，从而构建出复杂且高效的工作流。

三者合一，勾勒出的不再是单一的工具，而是一个坚实的基础设施框架——供开发者构建专属AI工作体系的基石。

咖哥将Claude Code形象地比作一座冰山，并将其画在白板上，如图1-1所示。



海平面之上，仅是露出一角，对应着小冰日常的使用场景：终端对话、代码生成与命令执行。而隐匿于海平面之下的庞大基座，则是一系列关键技术架构，它们精准地解决了小冰遭遇的困境。

- *CLAUDE.md* (记忆系统)：根治“失忆”顽疾，将项目规范一次性写入配置文件，即可在每次会话中自动加载，不需要反复重申。
- *Skills*：终结风格飘忽，将代码审查标准配置化、制度化，彻底取代“口头叮嘱”的不确定性。
- *子智能体*：化解上下文溢出，将涉及6个文件重构任务拆解为6个独立的上下文单元，实现并行处理与逻辑隔离。
- *Hooks*：在工具调用时自动触发安全检查或日志记录，构建防御性编程机制。
- *MCP* (Model Context Protocol, 模型上下文协议)：打破数据孤岛，赋予Claude调用外部数据库与API (Application Programming Interface, 应用程序接口) 的能力。
- *Headless模式*：支持在CI/CD流水线中“无人值守”运行，实现真正的自动化交付。
- *Agent SDK*：允许通过代码编排复杂的多步Agent工作流，提升任务执行的灵活性。
- *Plugins生态*：将上述能力打包封装，便于在团队内部高效分发与复用。

“你遭遇的每一个痛点，”咖哥指向冰山水下的庞大基座，“在这里都有对应的机制予以化解。问题不在于工具本身不够强大，而在于你尚未释放它真正的潜能。现在，我期望你能够登高望远，从全局视角重新审视并掌握Claude Code的全貌，将其威力彻底释放。”

小冰闻言，若有所思：“之前的用法，本质上是在进行低效的‘手动写脚本’——反复向Claude灌输相同信息、人工拆解任务、肉眼检查风格；而框架化的用法，则是通过恰当的配置将这一切转化为自动化的流程。”

咖哥点头确认：“没错。你每次开启新对话都要复述技术栈，每次代码审查都要手动描述标准，每次提交代码都依赖个人记忆去检查——这些本质上都是‘手动写脚本’式的低效操作。而框架化的用法则是：将技术栈固化于CLAUDE.md中，一劳永逸；将审查标准封装进Skill，实现自动加载；将提交检查配置为Hook，确保永不遗漏。”

这种从“逐次指令”向“系统配置”的范式转变，不仅极大地提升了效率，更深刻地重塑了人机协作的质量。配置一旦确立，Claude Code便不会遗忘、不会懈怠，更不会因深夜加班而降低标准——这正是将个体智慧沉淀为制度规范的工程思想精髓。

接下来，我们将深入拆解这一框架的内部结构。

1.2 四层架构模型

任何复杂系统在被人真正理解之前，都需要一张清晰的“地图”。正如建筑师在动工前需要绘制蓝图，软件工程师在编码前需要确立架构，Claude Code的完整技术栈亦可被抽象为一个四层架构模型，如图1-2所示。自下而上，这四层分别为：记忆层、扩展层、集成层和编程层。深入理解这四层的内涵及其边界，是掌握整个Claude Code体系的首要步骤。

1.2 四层架构模型

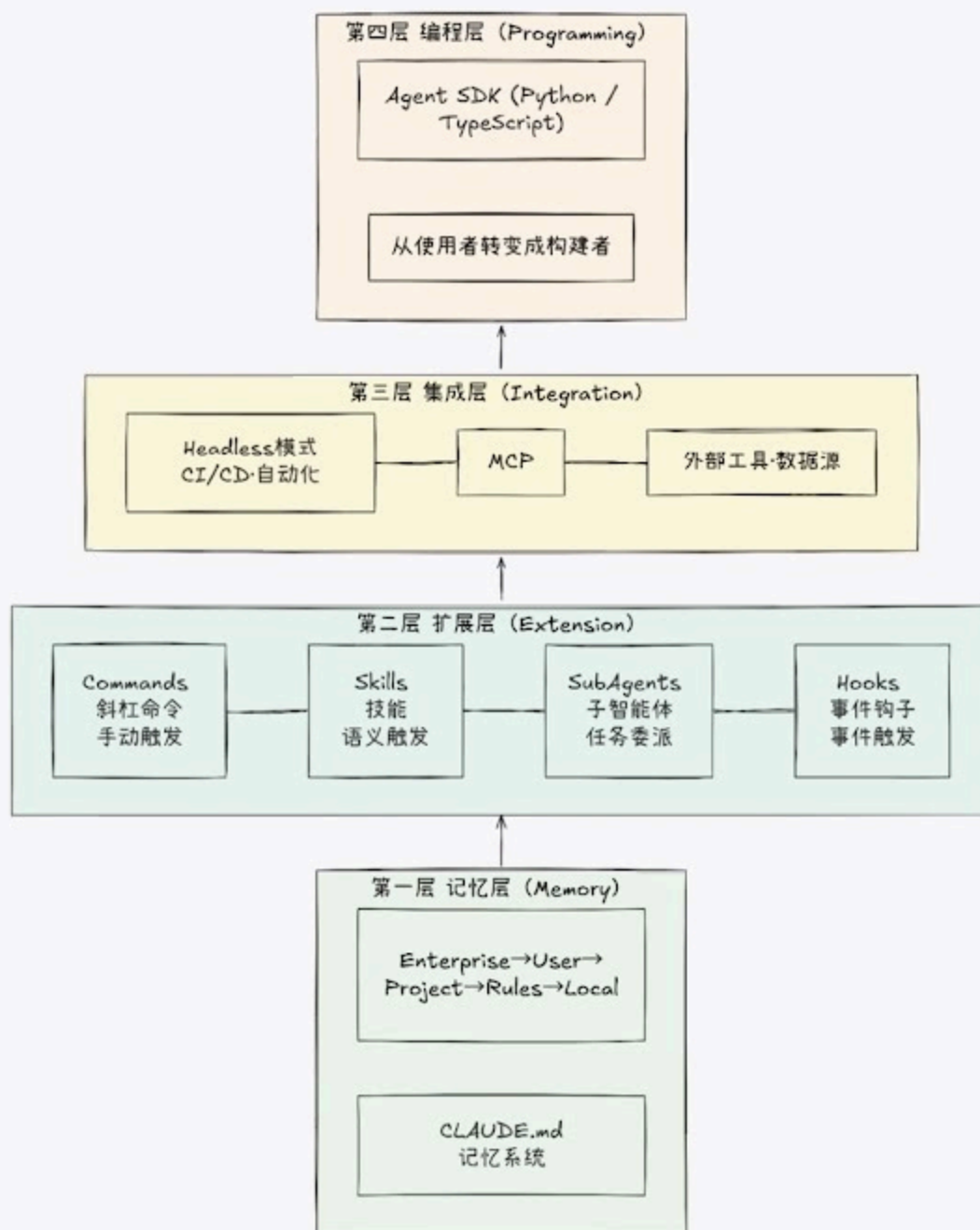


图1-2 Claude Code的四层架构模型

咖哥将这四层架构生动地比作一栋摩天大楼。

- 记忆层的记忆系统是深埋地下的地基。若无此根基，其上的一切构建都将无从谈起，随时面临坍塌风险。
- 扩展层的四大组件——Commands、Skills、SubAgents、Hooks，共同构成了建筑的主体楼层，承载着日常运营的核心功能，这是用户最直接感知的价值空间。
- 集成层的集成能力宛如隐藏起来却至关重要的水管网，它将这座建筑与外部广阔的基础设施网络紧密相连——通过连接广阔的基础设施，确保持续的能量与信息流动。

- 编程层的Agent SDK则是位于顶楼的建筑师工作室。在这里，你不再仅仅是住户，而是拥有了设计全新建筑、重构空间逻辑的“至高权力”。

“自下而上审视，是构建视角，关注基石与支撑；自上而下俯瞰，则是使用视角，聚焦功能与体验。”咖哥强调，“这句话值得铭记于心。”

1.2.1 记忆层——CLAUDE.md

建筑的地基决定了整栋大楼的承载上限。Claude Code的记忆层亦是如此——它直接定义了Claude对项目“理解”的深度。

本质上，Claude是一个无状态模型。每当开启新对话，它对项目背景、技术栈选择、代码规范及团队约定均一无所知。若未进行任何配置，开发者便需要反复重申诸如“使用TypeScript而非JavaScript”“优先选用pnpm而非npm”“测试框架定为Vitest”等基础设定。这种如同每日入职般重复自我介绍的模式，其低效程度可想而知。

记忆层通过CLAUDE.md彻底解决了这一痛点。CLAUDE.md本质上是一份“给AI的员工手册”，能在每次对话启动时自动加载至Claude的上下文中。更为精妙的是，Claude Code的这种记忆机制并非固定不变，而是构建了一个包含5个层级的记忆体系（见图1-3），每一级均对应不同的适用范围。

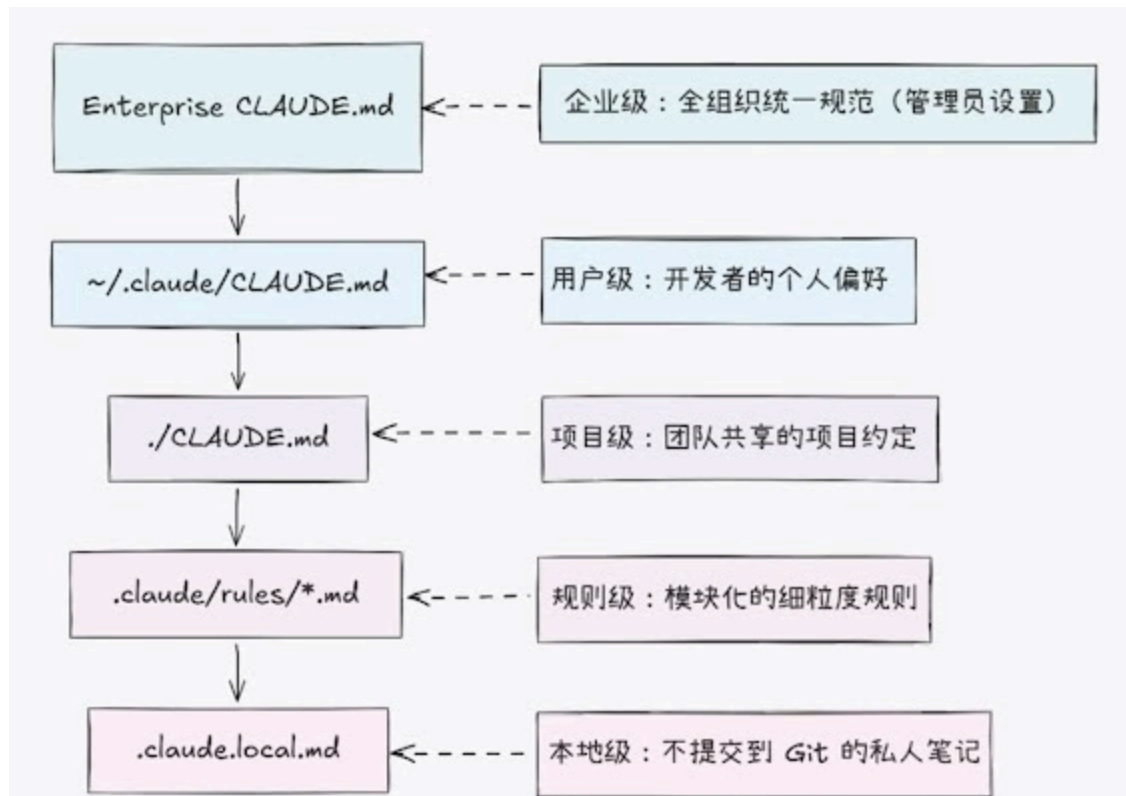


图1-3 Claude Code的5级记忆体系

这种分级设计背后的工程思想，与CSS的层叠优先级机制，或是软件配置中“全局—用户—项目—本地”的四级覆盖模式如出一辙：每一级均可覆盖上一级的设定，而层级越具体（如本地级），其优先级越高。

一个典型的项目级Claude.md示例如下：

```
# 项目记忆CLAUDE.md

## 技术栈
- 语言: TypeScript 5.x (启用严格模式)
- 框架: Fastify+Prisma

- 测试: Vitest (所有公共函数必须包含单元测试)
- 包管理: pnpm (禁用npm或yarn)

## 代码规范
- 组件风格: 优先使用函数式组件, 禁止使用class组件
- 错误处理: 统一采用Result类型返回, 严禁直接抛出异常
- 提交规范: 遵循<type>[optional scope]: <description>格式

## 常用命令
- pnpm dev - 启动开发服务器
- pnpm test - 运行测试套件
- pnpm lint - 执行代码检查
```

基于CLAUDE.md，Claude在每次开启对话时便会自动“研读手册”，从而避免技术栈误用与代码风格漂忽的风险。正如小雪所言：“自从创建了CLAUDE.md，我再也不必在每次对话中反复强调‘我们使用pnpm，而不使用npm’了。”

这一看似简单的机制，实则是整个框架的基石。上层的Skills、Commands、Hooks等所有配置与行为，均构建于Claude对项目上下文的精准理解之上。恰如建筑地基虽不如外墙装饰般醒目，但若缺了它，一切装饰都将无从依附。第2章将深入剖析这个看似朴素却至关重要的记忆机制。

1.2.2 扩展层——四大组件

若将记忆层比作地基，扩展层便是建筑的主体楼层——它赋予了Claude在日常使用中最丰富的能力。这一层由四大组件组成：Commands（斜杠命令）、Skills（技能）、SubAgents（子智能体）和Hooks（事件钩子）。它们既独立运作，又能灵活组合。

其中，**Commands**是最直观的扩展方式。当你在终端输入 /review、/commit 或 /deploy 等指令时，Claude便会依据预定义的模板，执行一套标准化的工作流。其本质是将高频重复、步骤固

定的操作封装为“一键触发”的快捷方式——这如同手机中的快捷指令，单次点击即可联动执行一系列预设动作。

从设计模式的角度审视，Commands正是命令模式(Command Pattern)的典型应用：它将“请求”封装为“对象”，从而使客户端能够以参数化的方式处理不同的请求。Commands的定义文件存放在 `.claude/commands/` 目录下，每一个Markdown文件即代表一条独立命令。例如，一个名为 `review.md` 的文件可包含代码审查的检查清单及输出格式规范；只需要输入 `/review`，Claude 便会严格遵照该清单，逐项对当前代码进行审查。



咖哥发言：截至本书完稿时，在Claude Code的最新演进中，Commands已正式并入Skills体系，被重新定义为一种“任务型Skill”——由用户通过 `/command` 触发的技能。本书在架构图中仍将其保留为独立模块，旨在强调其独特的触发范式：手动显式触发与语义自动触发的本质区别。深入理解这一区分，对于掌握整个扩展体系的设计思想仍有价值。

Skills遵循了截然不同的设计路径。它不需要用户手动触发，而是由Claude根据当前任务的语义上下文，自动研判并加载相应的技能。每个Skill本质上是一个包含 `SKILL.md` 的目录。在该文件中，通过YAML Frontmatter(前置元数据)声明skill的名称、描述及允许调用的工具列表。Claude正是依据 `description` 字段的内容，动态决策“当前任务是否需要激活此Skill”。这种隐式触发机制，使得Skills成为承载领域知识的理想载体——无论是财务分析方法、API设计规范，还是特定框架下的最佳实践，皆可封装其中。

例如，当你与Claude探讨财务指标时，它会自动加载“财务Skill”；而一旦话题转向代码架构，该Skill便会悄然退场。这种“按需加载、闲时静默”的机制，极大地提升了交互的流畅度与智能性。

以下是一个典型的skill.md示例：

```
---
name: code-reviewing
description: >
  Review code for best practices and potential issues.
  Use when the user asks for code review or mentions
  reviewing changes.
allowed-tools:
  - Read
  - Grep
  - Glob
---

# Code Review Guidelines
你是一名专业的代码审查员.....
```

子智能体旨在解决一个更深层次的瓶颈：上下文窗口的有限性。

Claude的上下文窗口犹如一张尺寸固定的办公桌：桌面空间有限，堆放的文件越多，越容易杂乱无章。当任务涉及海量文件（例如，审查整个PR的代码变更）时，若将所有细节强行塞入主对话的上下文，很快就会导致“桌面爆满”，致使Claude顾此失彼，甚至遗漏关键信息。

子智能体的解决方案是为特定任务开辟独立的上下文空间——这相当于在主桌旁支起一张新的工作台，指派专人在此处理子任务。待子任务完成后，仅需要将结论（一份简洁的报告）反馈至主桌面，而非将所有工作草稿全盘搬运。这就好比企业管理者向下属委派任务：管理者不需要深究每一行代码的细节，只需要关注审查结论与改进建议。

子智能体的定义文件位于 `.claude/agents/` 目录下，每个Markdown文件详细描述了一个子智能体的角色定位、权限范围及行为准则。更为关键的是，你可以为子智能体设置严格的工具权限，例如，规定“代码审查员”仅拥有读取文件的权限，严禁修改任何内容。这一设计深刻体现了最小权限原则(Principle of Least Privilege)在AI Agent架构中的核心应用。

Hooks是四大组件中唯一具备“拦截”能力的模块，如果说Commands定义了Claude“做什么”，Skills指导了Claude“怎么做”，那么Hooks则判断Claude“能不能做”。

它们在特定事件节点自动触发，包括工具调用前(PreToolUse)、工具调用后(PostToolUse)以及响应生成结束前(Stop)，开发者可在这些关键时机插入检查、拦截或增强逻辑。这一机制与Web框架中的中间件(Middleware)概念异曲同工。

例如，通过在 `.claude/settings.json` 中配置Hook，可实现：每当Claude试图执行 `git commit` 时，自动运行lint检查；如果检查未通过，则直接阻止提交操作。这种“隐形守卫”的角色，使Hooks成为保障代码质量与安全合规的利器。

以下是一个典型的配置示例：

```
{  
  "hooks": {  
    "PreToolUse": [  
      {  
        "matcher": "Bash",  
        "command": "python .claude/hooks/safety_check.py",  
        "blocking": true  
      }  
    ]  
  }  
}
```

上述配置的含义是：每当Claude准备调用bash工具执行命令时，系统优先运营安全检查脚本；如果脚本返回非零退出码（表示检查失败），此次调用将被强行阻断。

这种事件驱动的守护机制，在软件工程中有着成熟的对应物——无论是Git Hooks、数据库触发器，还是消息队列的消费者，其本质皆遵循统一模式：在特定事件发生时，自动执行预设逻辑。

这四大组件共同构筑了Claude Code扩展能力的核心基石。值得强调的是，它们之间呈现出正交性：既支持独立学习与单独使用，又能灵活组合以构建更强大的工作流。

这种正交设计深刻遵循了软件工程中的单一职责原则(Single Responsibility Principle, SRP)：每个组件专注解决一类特定问题（“只做好一件事”），而通过有机组合，却能激发出“无所不能”的系统效能。



咖哥发言：初学阶段，切勿急于求成，试图一次性掌握全部4个组件。建议遵循以下循序渐进的学习路径：首先，写好CLAUDE.md；其次，学习Commands，这是最直观的扩展方式；接下来，掌握Skills，体会语义自动触发的妙处；然后，引入Hooks，建立安全与质量的守护机制；最

后，挑战子智能体，以解决大规模复杂任务。每掌握一个组件，请务必在真实项目中实战演练两周，待融会贯通后再进入下一阶段。毕竟，“贪多嚼不烂”的道理，在此同样适用。

1.2.3 集成层——连接外部世界

在四层架构这幢摩天大楼中，集成层是维系其运转的“水电管网”——它将Claude Code的核心能力延伸至外部系统，使其真正融入开发生态。

其中，**Headless模式**是关键所在。它使Claude Code能够脱离交互式终端，在无人值守的环境中自动化运行。通过 `-p` 参数传入任务描述，并配合 `--output-format json` 规范输出格式，Claude Code即可无缝嵌入GitHub Actions、Jenkins、GitLab CI等任意CI/CD流水线。

一个典型的应用场景是自动化PR审查：每当开发者提交Pull Request，CI流水线便自动调用Claude Code进行代码审查，并将结果以评论形式直接回写至PR页面。整个过程不需要人工干预，真正实现了“无人值守”的自动化协作。

以下是在CI/CD流水线中调用Claude Code的示例命令：

```
# 在CI/CD流水线中调用Claude Code
claude -p "审查最近一次提交的代码变更，重点关注安全隐患与性能问题" \
  --output-format json \
  --max-turns 10 \
  --allowed-tools Read,Grep,Glob
```

上述命令的每个参数都蕴含着明确的工程意图：`-p` 用于激活Headless模式，实现非交互式运行；`--output-format json` 确保输出可被脚本自动解析；`--max-turns` 通过限制最大交互轮数来精准控制成本；而 `--allowed-tools` 则利用白名单机制严格限定Claude可调用的工具范围，从而筑牢安全防线。

MCP则开启了另一个维度的集成能力——它赋予Claude与外部工具及数据源深度交互的本领。如果说Headless模式是让Claude“走出去”以融入流水线，那么MCP则是让外部能力“走进来”以赋能模型。

只需要在项目根目录的 `.mcp.json` 文件中配置MCP服务器，即可轻松扩展Claude的能力边界：无论是访问数据库、调用第三方API，还是操作GitHub Issue、查询Jira工单，皆可一键打通。Anthropic将MCP形象地比喻为“AI时代的USB-C”——这是一套标准化的接口协议，使得任何工具和数据源都能以统一、规范的方式接入Claude Code生态。

以下是一个配置GitHub MCP服务器的示例

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
```

```

        "args": ["-y", "@modelcontextprotocol/server-github"],
        "env": {
            "GITHUB_TOKEN": "your-token-here"
        }
    }
}

```

MCP的设计理念值得深思。在传统AI工具集成中，每接入一个新的外部系统往往需要编写大量定制化的适配代码。而MCP通过定义一套标准化协议（涵盖Tools、Resources、Prompts三大核心能力），实现了工具提供者与AI消费者之间的“接口统一”。这与REST API在Web时代统一前后端通信的变革异曲同工——标准化带来的绝非限制，而是生态的繁荣。

Headless模式与MCP构成了集成层的两大支柱。

Headless模式面向自动化流水线，将Claude Code无缝嵌入既有的CI/CD体系。

MCP面向工具生态，将外部数据与服务引入Claude Code的能力工具箱。

二者合力，推动Claude Code从一个独立运行的终端工具，进化为可灵活融入任何技术栈的基础组件。如果说扩展层是在Claude Code的“体内”强化其核心机能，那么集成层便是在“体外”构建连接网络——两者相辅相成，缺一不可。

1.2.4 编程层——Agent SDK

摩天大楼的顶楼是建筑师的工作室——在这里，你不再是栖身其中的住户，而是设计新蓝图的创造者。

Agent SDK位于Claude Code技术栈的顶端，是一道关键的“分水岭”。在此之下的三层（记忆层、扩展层和集成层），开发者主要通过配置文件和命令行来延展Claude的行为边界，其本质上是使用者。而一旦跨越Agent SDK这条界线，便正式踏入构建者的领地：开发者可以利用Python或TypeScript编写代码，直接调用Claude Code的底层核心能力，从零开始构建全新的AI Agent。

以下是利用Agent SDK构建一个代码健康度来检查Agent的示例：

```

import claude_code

# 用SDK构建一个代码健康度检查Agent
result = claude_code.query(
    prompt="分析src/目录下所有Python文件的代码质量，给出健康度评分",
    allowed_tools=["Read", "Grep", "Glob"],
    max_turns=15
)

print(result)

```

上述代码的深层意义：开发者不再受限於Claude Code的终端交互界面，而是能够在自己的应用程序中直接驱动Claude执行任务。这意味着，Claude的分析与推理能力可以无缝嵌入到定时任务、Web服务、数据处理管道等任何场景中——开发者的代码能触达之处，便是Claude能力延伸之所。

对于绝大多数开发者而言，前三层（记忆层、扩展层、集成层）已足以从容应对日常开发场景。然而，如果你的愿景是构建团队级AI开发平台，或者设计多Agent协作的自动化系统，那么Agent SDK便是必经之路。

借助Agent SDK，你可以打造批量的代码审查工具、项目健康度仪表盘、自动化测试报告生成器，甚至将多个Claude Agent编排成一个高效的协作团队，例如，让第一个Agent专司代码阅读，第二个Agent负责编写测试，第三个Agent负责执行审查，三者串联成一条全自动化的智能流水线。

Agent SDK的存在还揭示了一个耐人寻味的事实：Claude Code本身，正是基于Agent SDK构建的一个AI Agent。你在终端中所见的一切行为——读取文件、编写代码、执行命令——本质上都是Agent SDK层面的工具调用。一旦洞察此点，你对Claude Code的认知便将从“一款产品”升维为“一种架构模式的具体实现”。

在后续章节中，我们将深入剖析Agent SDK的实战用法。在这里，我们只需要确立一个核心认知：Agent SDK是这栋架构大楼的顶层，更是从“配置驱动”跃迁至“代码编程”的起点。

1.2.5 底层视角：Harness与Agentic Loop

在理解了四层架构的“外部视角”后，我们不妨深入底层，探究一个更为本质的概念——**Harness**（马具）。

Anthropic官方文档对此的定义是：“Claude Code serves as the **agentic harness** around Claude: it provides the tools, context management, and execution environment that turn a language model into a capable coding agent.”

简而言之，Claude Code是一个包裹在Claude模型外层的Agent编排框架。原生模型仅具备生成文本的能力。正是Harness赋予了它读取文件、编写代码、检索代码库以及在终端执行命令的“手脚”。若缺失了Harness，Claude不过是一个空有智力却寸步难行的“大脑”。

Harness一词的原意为“马具”，即套在马身上的挽具与缰绳。马匹虽有力气，但若无马具的引导，便无法拉动车辆。Harness改变的并非马的力量本身，而是力量的传导与控制方式。

若用公式来表达，即 **Agent=Model + Harness**。

如果把模型比作引擎，Harness则是围绕引擎构建的一切基础设施：工具系统、权限控制、上下文管理、会话持久化、事件钩子，以及驱动这一切运转的核心循环——**Agentic Loop**（见图1-4）。

这一循环机制深刻揭示了Claude Code核心能力的来源。当你提交一个bug时，Claude Code并非简单地“看一眼便猜测答案”。而是经历了一个反复观察、假设与验证的过程：先查阅报错日志，继而搜索相关代码，深入理解上下文，最后才动手修复。

Agentic Loop

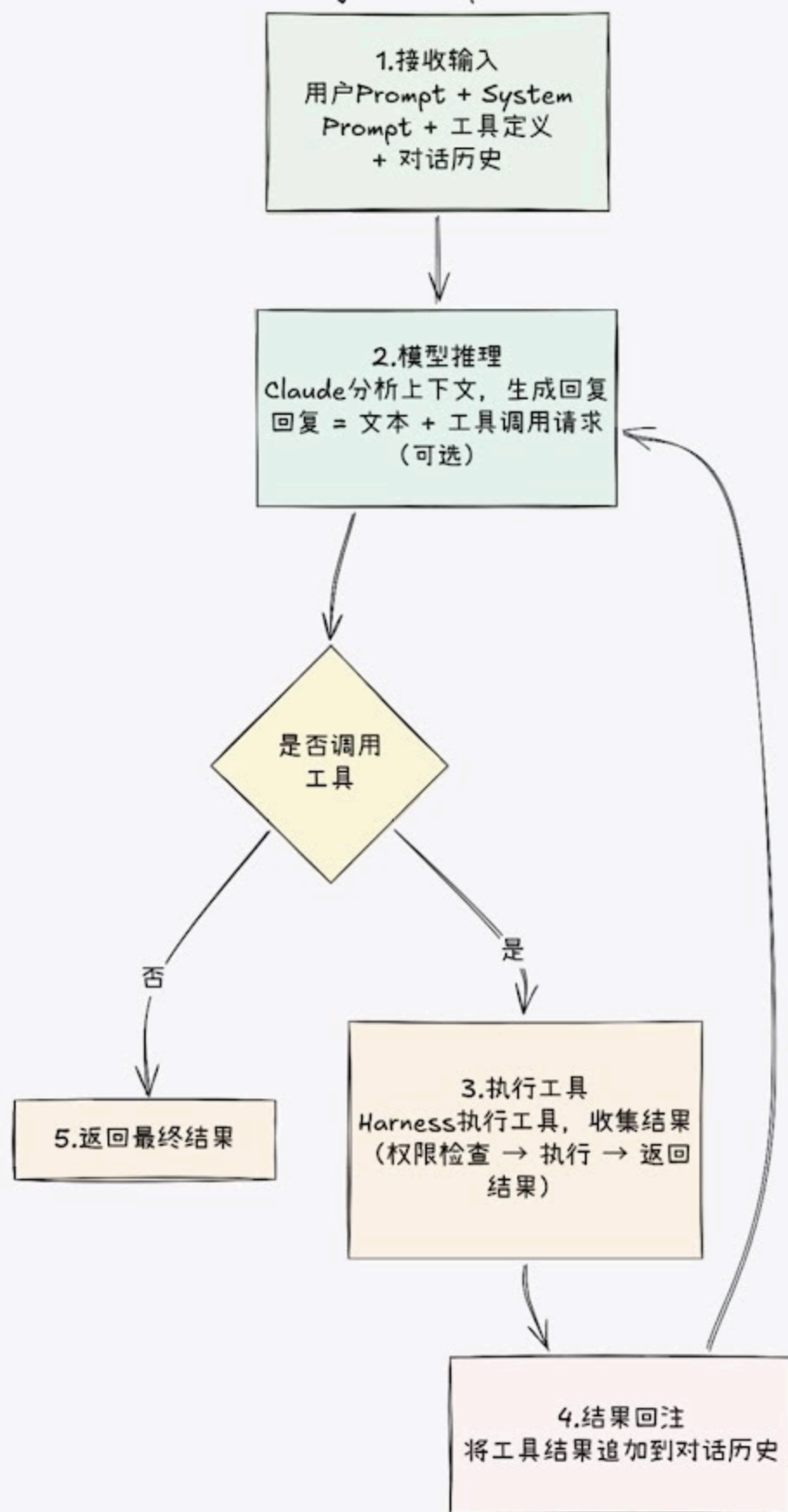


图1-4 Agentic Loop——Harness的心脏

循环何时终止？取决于两个条件：一是模型主动停止（即认为任务已完成，不再发起工具调用）；二是达到最大轮次限制（由 `--max-turns` 参数设定，旨在防止程序陷入无限循环）。

2026年，行业涌现出一个关键洞察：**同一模型在不同Harness下的表现差异，远大于不同模型在同一Harness下的表现差异。**

在TerminalBench基准测试中，仅通过对Harness的优化，就使同一模型从基线水平以下跃升至Top 5行列。换句话说，Harness比模型本身更为重要。你之前所学的四层架构中的每一环，无论是CLAUDE.md的撰写、工具权限的设置、Hooks的接入，还是MCP的连接，本质上都是在对**Harness**进行精雕细琢。

小冰听罢恍然大悟：“我之前以为Claude Code仅仅是Claude模型的一个外壳。原来，这个‘外壳’才是其真正的价值所在。”

1.3 组件关系与协作

在深入认识四层架构的每个组成部分后，更关键的问题在于：它们之间如何协同工作？在不同场景下应如何选择使用？一个真实的任务在该体系下又是如何流转的？

1.3.1 触发机制对比

扩展层的四大组件最易混淆之处在于其触发机制，表1-1从5个维度进行了对比。

组件	触发方式	触发者	是否需要记忆	典型场景	工程类比
Commands	用户手动输入	人	是（需要知晓命令名...	<code>/review</code> 、 <code>/deploy</code>	CLI 命令
Skills	Claude 基于语义自动匹配	AI	否（自动发现）	财务分析、API 设计	策略模式（自动选择算法）
子智能体	用户指定或 Claude 主动委派	人或 AI	是（需要知晓 Agent 名称）	代码审查、日志分析	线程池（隔离执行）
Hooks	系统事件自动触发	系统	否（配置即生效）	提交前检查、保存后格式化	中间件 / Git Hooks

直观的记忆口诀如下。

- *Commands*：“你叫它做”（响应用户指令）。
- *Skills*：“它自己知道该做”（自主判断并执行）。
- *子智能体*：“它安排别人做”（协调子智能体完成任务）。

- *Hooks*: “不管谁做，到了这一步就执行检查”(在特定流程节点进行自动拦截或处理)。

小雪看到上述总结后，精辟地指出：“Commands和Hooks均属于确定性触发——前者由用户指令明确触发，后者由特定事件自动触发；而Skills和子智能体则体现了AI的自主判断——前者由AI判断是否启用，后者由AI判断是否进行任务委派。”

咖哥赞许地点头道：“这种‘确定性与AI判断’的二分法，确实是理解四大组件差异的另一个关键视角。在实际工程中，对于安全性要求极高的场景（如敏感信息拦截），应当优先采用确定性触发的Hooks；而对于灵活性要求较高的场景（如领域知识匹配），则更适合使用基于AI判断触发的Skills。”

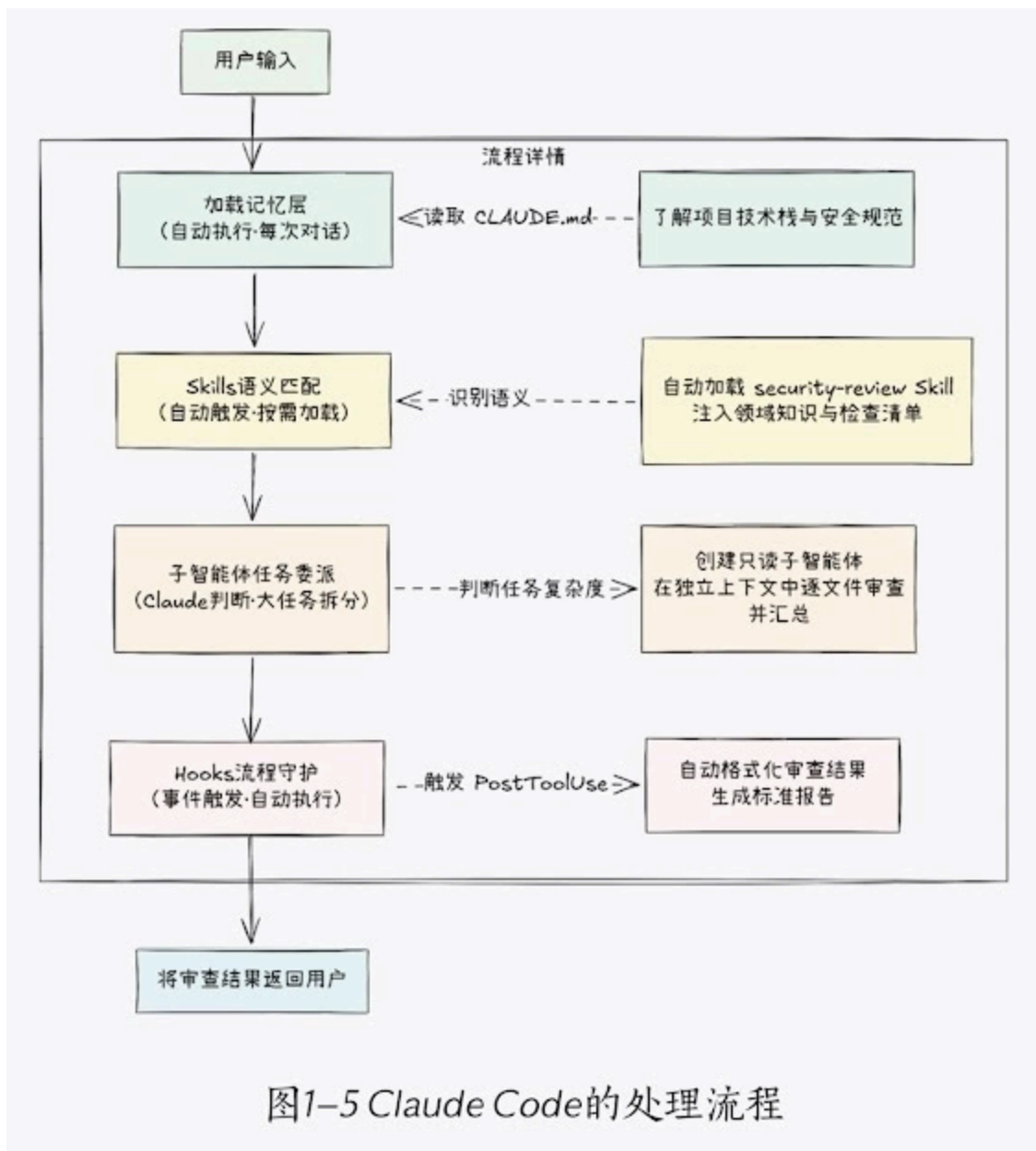
1.3.2 数据流：一个请求的旅程

为了更直观地理解各组件的协同机制，让我们追踪一个请求在Claude Code体系中的完整流转路径。

假设开发者在终端输入指令：“帮我审查src/payment/目录最近的代码变更，确保没有安全隐患。”

Claude Code的处理流程如图1-5所示。

在图1-5所示的流程中，记忆层和扩展层中的各个组件各司其职：记忆层(CLAUDE.md)构建项目上下文，Skills注入领域知识，子智能体提供隔离执行环境，Hooks则严守质量关卡。全程不需要用户额外干预——一切均在用户输入那句自然语言指令之后自动编排完成。



这正是“可组合性”的真谛。单个组件如同独立的积木，一旦组合，便形成了一条完整的自动化流水线。这完美契合了软件工程中经典的“关注点分离”(Separation of Concerns)原则：每个模块专注单一职责，通过清晰的接口协同工作。Claude Code的组件设计正是这一原则的典范。

- *CLAUDE.md*专注于“知道什么”(Know-what)。
- *Skills*专注于“怎么做”(Know-how)。
- 子智能体专注于“谁来做”(Who-does-it)。
- *Hooks*专注于“能否做”(Guard-rails)。

正因职责边界清晰，这些组件才能自由组合而互不掣肘。

值得注意的是，上述案例展示了一种“全家桶”式的协作场景。在实际工程中，大多数任务并不需要涉及所有组件：简单的代码生成可能仅需要记忆与Skills的配合，日常提交操作或许只涉及Hooks。组件的核心价值不在于“全部用上”，而在于“按需调用，随取随用”。

1.3.3 Plugins: 组合的打包与分发

“小冰，你曾问我：‘Skills与Plugins究竟有何关联？’”咖哥在白板上画出一个大盒子，然后揭晓了答案，“很简单——Plugins并非新的能力，而是一种打包机制。”

试想，你耗时3周为团队精心构建了一套Claude Code扩展体系：包含两个Skill（代码审查、API设计）、一个子智能体（安全审查员）以及3个Hook（提交前Lint检查、保存后格式化、敏感信息拦截）。如今新同事入职，急需复用这套配置。如果采用手动复制文件的方式，光是厘清“哪些文件该放入哪个目录”就足以让人耗费半天精力。

Plugins机制优雅地解决了这一难题。它本质上是一种标准化的打包格式：通过一个 `plugin.json` 清单文件，将Skills、Commands、子智能体、Hooks及MCP配置统一封装。打包完成后，只需要一条安装命令，所有配置即可自动部署到位。

这就好比npm之于Node.js、Docker之于容器镜像——Plugins的核心价值在于分发与复用，而非创造新的能力维度。

厘清这一概念至关重要。初学者常误将Plugins视为“更高级的Skills”，实则二者处于完全不同的维度：Skills定义的是能力类型，而Plugins定义的是分发形式。

一个Plugin如同一个容器，内部可封装Skills、Commands、Hooks及子智能体等多种组件。它关注的核心并非“执行什么任务”，而是“如何打包与共享”。

对个人开发者而言，或许不需要刻意创建Plugins，直接在项目本地配置各组件即可满足需求。

对团队负责人而言，Plugins则是将最佳实践标准化、实现新人入职“一键配置到位”的利器。

1.4 技术选型指南

面对纷繁的组件，一个核心问题随之而来：针对手头的具体需求，究竟该选用哪一个？

为此，这里提供了Claude Code组件选型决策树（见图1-6）和应用场景对比（见表1-2），助你快速做出判断。

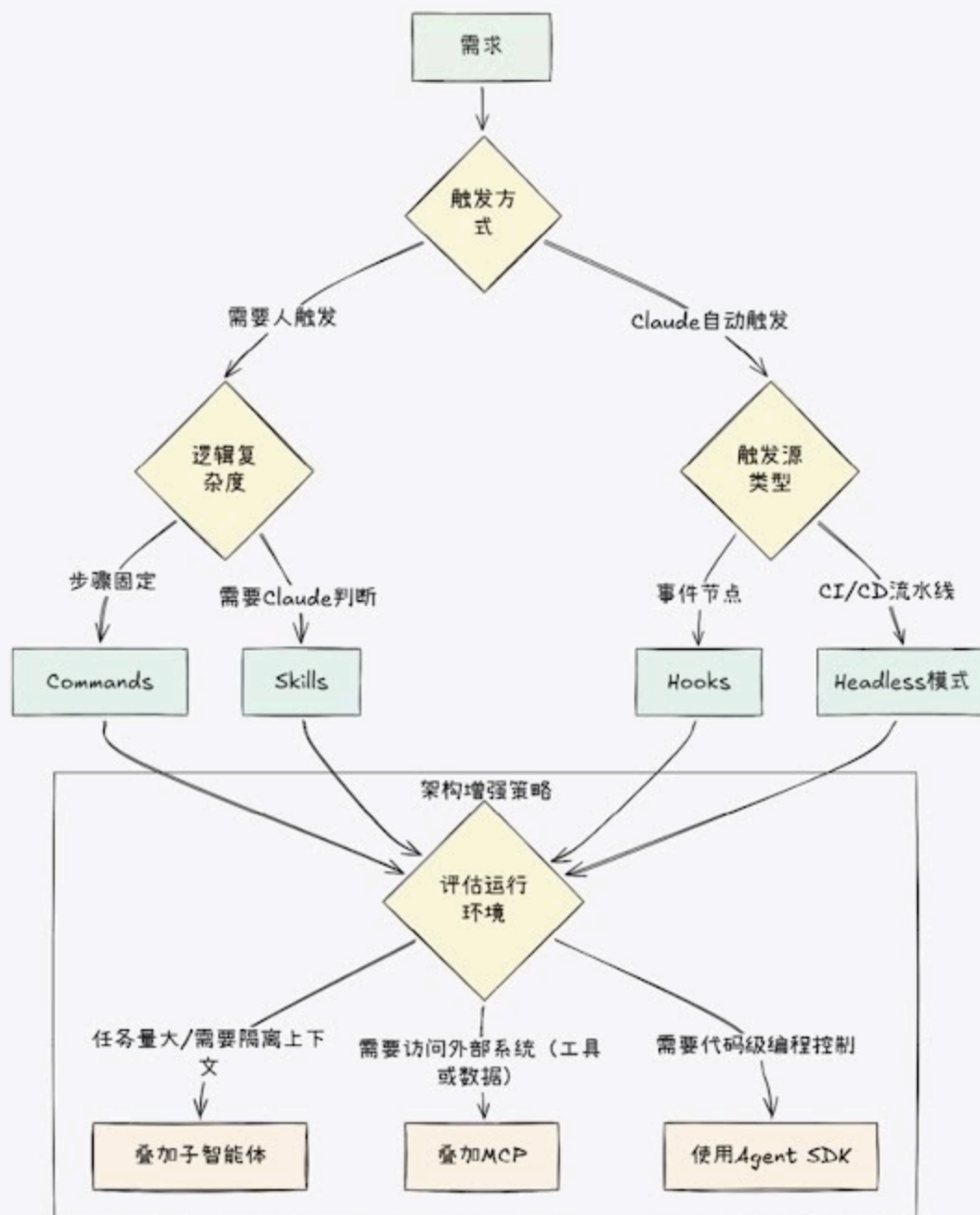


图1-6 Claude Code组件选型决策树

应用场景	推荐组件	核心理由
每次发布新版本时执行固定的检查列表	Commands	步骤明确且固定，适合通过手动指令一键触发
讨论财务问题时自动获得专业指导	Skills	基于语义自动匹配领域知识，按需加载，不需要用户手动指定或记忆命令
审查大型 PR，避免上下文溢出	子智能体	利用于智能体隔离上下文，仅向主会话返回精简结论，有效管理 Token（词元）消耗

应用场景	推荐组件	核心理由
提交代码前自动阻止敏感信息泄露	Hooks（PreToolUs...	事件驱动机制，确保在特定操作（如工具调用）发生前强制拦截检查，无遗漏
PR 提交后自动触发代码审查	Headless 模式 + Commands	适用于无人值守环境，可嵌入 CI/CD 流水线，通过命令自动执行审查任务
让 Claude 操作 GitHub Issue	MCP	通过标准协议无缝接入外部系统（如 GitHub API），扩展模型的工具边界

针对决策树，请按照以下路径进行自我提问。首先，问自己“这个任务的触发方式是什么？”如果需要人触发，则继续问“逻辑复杂程度如何？”如果步骤固定，则选择Commands；如果需要Claude判断，则选择Skills。如果是Claude自动触发，则继续问“是在特定事件节点触发，还是在CI/CD流水线中触发？”如果是事件节点，则选择Hooks；如果是CI/CD流水线，则选择Headless模式。其次，问自己“运行环境如何？”如果任务量大或需要隔离上下文，则叠加使用子智能体。如果需要访问外部系统（如工具或数据），则叠加使用MCP。如果需要进行代码级编程控制，则使用Agent SDK。

值得注意的是，这些组件在实际使用中往往是叠加协作而非互斥单选的。

一个成熟的团队配置通常会统筹所有组件，构建起严密的自动化体系。

- *CLAUDE.md*奠定基础上下文。
- *Skills*注入领域专业知识。
- *Commands*提供标准化的人工操作入口。
- *Hooks*构筑自动化的安全防线。
- 子智能体拆解并处理高复杂度任务。
- *MCP*打通外部系统的数据壁垒。
- *Headless*模式则将能力无缝嵌入CI/CD流水线。

这正如一家运转高效的企业：每个角色（组件）各司其职、边界清晰，却又紧密协同，共同达成最终目标。

咖哥发言：技术选型的核心原则始终是“用最简单的方案解决当前问题”。切勿陷入“手里有锤子，看什么都是钉子”的误区：不要刚学会子智能体就处处委派任务，也不要刚掌握Hooks就给每个微小操作都加上守卫。过度工程化与工程化不足同样有害。优秀的架构师深知平衡之道：既明白何时需要引入一层抽象以提升扩展性，也清楚何时仅需要CLAUDE.md便足以完美地解决问题。

本章小结

本章始于一个关键的认知转变：Claude Code并非命令行版的ChatGPT，而是一个可编程、可扩展、可组合的Agent框架。就Claude Code这座冰山而言，水面之上的对话交互仅是入口，深藏

水面之下的记忆系统、四大组件、集成机制及Agent SDK，才构成了其真正的技术纵深。理解这一点，是从“被动使用”迈向“主动编排”的第一步。

四层架构为整个体系绘制了一张清晰的蓝图。记忆层是地基，赋予Claude Code理解项目上下文的能力；扩展层是主体，提供Commands、Skills、子智能体、Hooks这4种具有正交性的扩展方式；集成层相当于“水电管网”，通过Headless模式和MCP连接外部世界；编程层则是“顶楼”，助力开发者从使用者跃迁为构建者。这四层架构层层递进，却也支持独立研习——你不需要通盘掌握所有内容，即可开启实践之旅。

厘清组件之间的关系，是本章最需要内化的核心知识。Commands代表显式指令，Skills承载隐式知识，子智能体实现隔离委派，Hooks则充当自动守护。这4种触发机制覆盖了从人工干预到自动执行、从显式调用到隐式响应的完整光谱。它们既可独立运作，亦能组合编排。而Plugins作为标准化封装机制，使得这些组合方案得以高效分发与复用。

在第2章中，我们将深入四层架构的基石——记忆系统(CLAUDE.md)，它既是整幢大楼的地基，也是你实践Claude Code框架化能力的最佳起点。一份精心撰写的CLAUDE.md，胜过10次冗余的口头说明。

思考题

1. 回顾你目前使用Claude Code（或其他AI编程工具）的习惯：哪些操作仍停留在“冰山之上”的被动交互层面？如果引入本章所述的组件，其中哪些环节可实现自动化或结构化重构？
2. 在团队的日常开发流程中，哪些环节最适合通过Hooks进行自动守护（如代码规范检查、敏感信息拦截）？哪些环节更适合固化为Commands以供显式调用？请列举3~5个具体场景，并为每个场景匹配最恰当的组件。
3. 本章指出，Claude Code的四大组件遵循“单一职责原则”。请深入思考：倘若将Skills与Commands合并为单一组件（既支持手动触发，又支持自动响应），这将带来何种便利？又会引发哪些潜在问题？这一设计背后，折射出软件设计中“简单性”与“正交性”的经典权衡。